| | |
|---|---|
| **FORMATO PARA ELABORACIÓN DE TALLERES ESTUDIANTILES** | Departamento: Ciencias de la Computación |
| | Carrera: ITIN Asignatura: Metod De Desarrollo Software NRC: 30693 |
| | Apellidos y nombres del estudiante: Sánchez Andrade Vicente Mateo |

## ANTECEDENTES

Como parte del estudio de la asignatura de metodologías de desarrollo de software, hemos desarrollado este trabajo para profundizar en el estudio que realiza la ingeniería de software. La resolución de este cuestionario permite analizar los fundamentos, procesos y paradigmas técnicos necesarios para estructurar proyectos de manera profesional. El objetivo es interiorizar cómo estas metodologías garantizan la calidad y eficiencia en la creación de sistemas, preparándonos para enfrentar los retos reales de la industria.

## OBJETIVO

Profundizar en el campo de acción de la ingeniería de software y sus fundamentos teóricos para comprender las metodologías de desarrollo de software desde la perspectiva de la Ingeniería de Tecnologías de la Información.

Identificar las actividades fundamentales del proceso de software, incluyendo la comunicación, planeación, modelado, construcción y despliegue.

## DESARROLLO

### Capítulo 1

### Actividad de aprendizaje 1

**1. ¿Qué es ingeniería de software?**

Es una disciplina de la ingeniería que se ocupa de todos los aspectos de la producción de software, desde las etapas iniciales de especificación hasta el mantenimiento después de la puesta en marcha

**2. ¿Cuáles son las actividades fundamentales de la ingeniería de software?**

Según Pressman:

- Comunicación: Colaboración con el cliente
- Planeación: Estimación, programación y seguimiento
- Modelado: Análisis y diseño
- Construcción: Generación de código y pruebas
- Despliegue: Entrega y retroalimentación

**3. ¿Cuál es la diferencia entre ingeniería de tecnologías de la información y ciencias de la computación?**

- Ciencias de la Computación: Según Sommerville se centran en las teorías y fundamentos que subyacen a las computadoras y los sistemas de software (teoría de algoritmos, estructuras de datos)
- Ingeniería de Tecnologías de la Información: Se enfoca en la selección, creación, aplicación, integración y administración de tecnologías computacionales para satisfacer las necesidades de los usuarios y organizaciones dentro de un contexto social y empresarial

**4. ¿Qué es software?**

Para Pressman, el software es un paquete integral, si falta la documentación el software es difícil de mantener si faltan las estructuras de datos, el programa no tiene qué procesar y si faltan las instrucciones, no hay ejecución

**5. ¿Cuáles son los atributos del buen software?**

Para que el software sea profesional, debe poseer:

- Mantenibilidad: Capacidad de evolucionar ante cambios.
- Confiabilidad y Seguridad: No causar fallos dañinos y resistir intrusiones.
- Eficiencia: Uso óptimo de recursos (memoria, procesador).
- Aceptabilidad (Usabilidad): Debe ser comprensible para el usuario final.
- Facilidad de prueba (Pressman): La capacidad de verificar errores fácilmente.

**6. ¿Cuáles son los principales retos que enfrenta la ingeniería de software?**

- Heterogeneidad: Construir software para redes distribuidas con dispositivos y plataformas diversas.
- Cambio constante: La necesidad de reducir tiempos de entrega (Time-to-market) sin degradar la calidad.
- Confianza: Garantizar la seguridad cibernética en un mundo donde la infraestructura crítica depende del software.

**7. ¿Cuáles son los costos de la ingeniería de software?**

De acuerdo con Sommerville, no existe una cifra absoluta, pero la distribución del esfuerzo y el dinero sigue patrones claros:

- Desarrollo vs. Pruebas: Aproximadamente el 60% de los costos se destina al desarrollo (especificación, diseño y programación) y el 40% restante a las pruebas (validación y verificación).
- Mantenimiento y Evolución: Para los sistemas de larga duración, el costo de mantener y evolucionar el software una vez que ya está en uso casi siempre supera con creces el costo de su desarrollo inicial.
- Dependencia del tipo de sistema: Si se desarrolla un sistema crítico (donde un fallo puede costar vidas o mucho dinero, como software médico o aeroespacial), los costos de validación y pruebas aumentan drásticamente, superando el 50% del presupuesto.

**8. ¿Cuáles son los mejores métodos y técnicas de la ingeniería de software?**

Para Pressman, la ingeniería de software no se basa en un método único, sino que es una "tecnología estratificada" (en capas). Los mejores métodos surgen de aplicar esta estructura de cuatro niveles:

1. Un enfoque de calidad: Es la base; cualquier técnica debe buscar la calidad total del producto.
2. El proceso: Es el marco de trabajo que define cómo se entregan los métodos y herramientas de forma oportuna.
3. Los métodos: Proporcionan las técnicas de "cómo hacer" las tareas específicas (análisis de requisitos, diseño, codificación y pruebas).

4. Las herramientas: El uso de herramientas CASE para automatizar y dar soporte a los procesos y métodos siempre que sea posible.

**9. ¿Qué entiende por paradigmas de la ingeniería de software?**

Sommerville se refiere a los paradigmas habitualmente como "Modelos de procesos de software".

Los define como una representación simplificada o abstracta de un proceso de desarrollo de software, que se presenta desde una perspectiva específica. No son descripciones definitivas de cómo debe hacerse el software, sino enfoques o estrategias arquitectónicas sobre cómo organizar las actividades (especificación, desarrollo, validación y evolución). Ejemplos clásicos incluyen el modelo en Cascada, el desarrollo iterativo y la ingeniería de software basada en componentes.

**10. ¿Cuál es la definición del modelo incremental versus otros paradigmas?**

Según Pressman, el modelo incremental es un paradigma que aplica secuencias lineales de forma escalonada, produciendo en cada ciclo un "incremento" de software entregable y operativo.

- Diferencia con el modelo en cascada: La cascada asume que los requisitos están completos desde el inicio y entrega el producto final de una sola vez. El modelo incremental asume que los requisitos pueden evolucionar y entrega un producto central funcional (el núcleo) rápidamente en el primer incremento.
- Diferencia con los prototipos: En la construcción de prototipos, el sistema inicial suele desecharse una vez que se entienden los requisitos. En el modelo incremental, cada entrega es un producto real, estable y utilizable que se integra al sistema final.

**Características y diferencias con otras ingenierías.**

Para Sommerville, la ingeniería de software tiene características únicas que la diferencian de la ingeniería tradicional (como la civil o la mecánica):

- Intangibilidad y ausencia de límites físicos: El software es un concepto lógico, no físico. Los ingenieros civiles están limitados por las propiedades físicas de los materiales (gravedad, resistencia). En el software no existen estos límites naturales, lo que permite crear sistemas de una complejidad abrumadora, haciendo que su control y visualización sean mucho más difíciles.
- El deterioro por evolución: El software no se estropea físicamente con el uso ni el clima (no se desgasta). Sin embargo, a diferencia de un puente o un motor, el software se degrada o deteriora lógicamente a medida que sufre constantes modificaciones y actualizaciones para adaptarse a nuevos requisitos, lo que vuelve su estructura interna cada vez más frágil y compleja.
- Relación con la Ciencia de la Computación vs. Sistemas: Sommerville aclara que la Informática o Ciencia de la Computación trata la teoría y los fundamentos, mientras que la Ingeniería de Software se ocupa de los aspectos prácticos de la construcción de sistemas útiles y entregables.

## Capítulo 2
## Autoevaluación

**1. ¿Qué es un conjunto de métodos que cubren todo el ciclo de vida de desarrollo de sistemas y que están unidos por un enfoque general filosófico?**

- Un conjunto de lenguajes de programación que permiten analizar, diseñar y construir productos software.
- Un conjunto de rutinas de programación que permiten desarrollar aplicaciones de forma ágil.
- Un conjunto de métodos que cubren todo el ciclo de vida de desarrollo de sistemas y que están unidos por un enfoque general filosófico.

**Respuesta correcta:** Un conjunto de métodos que cubren todo el ciclo de vida de desarrollo de sistemas y que están unidos por un enfoque general filosófico. Esto define una metodología de desarrollo de software, que incluye principios, prácticas y fases desde la concepción hasta el mantenimiento. (Pressman, 2010)

**2. Las actividades estructurales del proceso de desarrollo de software son:**

- Despliegue / comunicación / modelado / diseño.
- Gestión de la configuración / modelado.
- Administración del proyecto / diseño.

**Respuesta correcta:** Despliegue / comunicación / modelado / diseño. Estas actividades representan las fases clave en el desarrollo estructurado de software. (Sommerville, 2011)

**3. Una de las razones por la cual se incrementan los costos al agregar funcionalidad después que un sistema está en operación es:**

- Lenguaje de programación seleccionado.
- Edad de los programadores.
- Práctica de desarrollo deficiente.

**Respuesta correcta:** Práctica de desarrollo deficiente. Una mala planificación o codificación inicial genera dificultades al modificar el sistema, lo que incrementa los costos. (Kerzner, 2013)

**4. Ingeniería de software asistida por computador significa:**

- Conjunto de herramientas de software integradas para ser utilizadas por otras.
- Conjunto de herramientas de hardware integradas para ser utilizadas por otras.
- Elementos físicos del computador integrados para ser utilizados por otras.

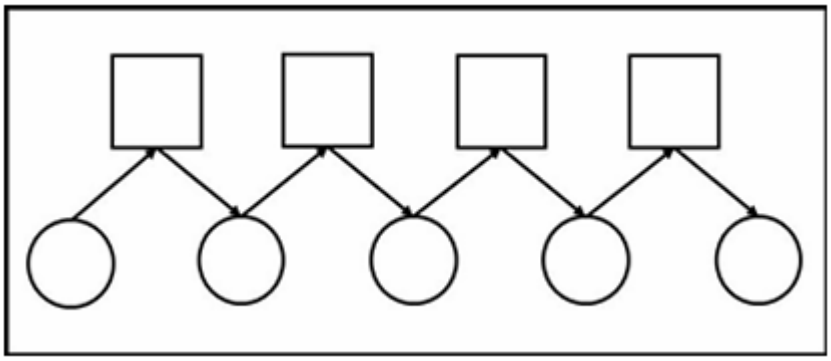
**Respuesta correcta:** Conjunto de herramientas de software integradas para ser utilizadas por otras. Se refiere a CASE (Computer-Aided Software Engineering), que incluye herramientas para análisis, diseño, codificación y pruebas. (Pressman, 2010)

**5. La metodología con sus tipos correspondientes de clasificación es:**

- Metodología Scrum (estructurada).
- Metodología XP (orientada a objetos – ágil).
- Proceso unificado de desarrollo (estructurada).

**Respuesta correcta:** Metodología XP (orientada a objetos – ágil). XP (Extreme Programming) es una metodología ágil basada en principios de la programación orientada a objetos y ciclos iterativos cortos. (Beck, 2005)

**6. En el gráfico propuesto, la secuencia ordenada de las figuras es:**



1. Necesidad, obtener requisitos, diseñar sistema, especificación de requisitos, diseño, codificar, código, probar, sistema de software.
2. Obtener requisitos, diseñar sistema, especificación de requisitos, diseño, codificar, código, probar, sistema de software, necesidad.
3. Necesidad, especificación de requisitos, obtener requisitos, diseñar sistema, diseño, codificar, código, probar, sistema de software.

**Respuesta correcta:** Necesidad, especificación de requisitos, obtener requisitos, diseñar sistema, diseño, codificar, código, probar, sistema de software.

**7. Los modelos de calidad del producto software y los modelos de calidad del proceso de desarrollo de Software son:**

- ISO/IEC 25010 producto software.
- ISO/IEC 20000 producto software.
- ISO 9126 proceso de desarrollo.

**Respuesta correcta:** ISO/IEC 25010 producto software. La norma ISO/IEC 25010 define un modelo de calidad para productos software, incluyendo características como funcionalidad, fiabilidad, usabilidad, seguridad, entre otras. La ISO/IEC 20000 se enfoca en gestión de servicios de TI, y la ISO 9126 fue reemplazada por la 25010 (ISO, 2011).

**8. La norma ISO/IEC 25010 identifica características y subccaracterísticas de calidad del producto software. La opción correcta es:**

- Seguridad es subcaracterística.
- Eficiencia de desempeño es característica.
- Fiabilidad es subcaracterística.

**Respuesta correcta:** Eficiencia de desempeño es característica. Según ISO/IEC 25010, la eficiencia de desempeño es una característica principal del producto software. Dentro de ella hay subccaracterísticas como tiempo de respuesta, utilización de recursos y capacidad (ISO, 2011).

**Bibliografía**


- Beck, K. (2005). *Extreme Programming Explained: Embrace Change* (2ª ed.). Addison-Wesley.
- ISO. (2011). *ISO/IEC 25010:2011 Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models*. International Organization for Standardization.
- Kerzner, H. (2013). *Project Management: A Systems Approach to Planning, Scheduling, and Controlling* (11ª ed.). Wiley.
- Pressman, R. S. (2010). *Ingeniería del software: Un enfoque práctico* (7ª ed.). McGraw-Hill.
- Project Management Institute (PMI). (2017). *A Guide to the Project Management Body of Knowledge (PMBOK Guide)* (6ª ed.). PMI.
- Sommerville, I. (2011). *Ingeniería de software* (9ª ed.). Pearson Educación.

## CONCLUSIONES

1. Se concluye que la ingeniería de software es una disciplina integral que abarca desde la especificación inicial hasta el mantenimiento post-implementación, diferenciándose de las ciencias de la computación al enfocarse en la aplicación práctica y organizada para satisfacer necesidades empresariales.

2. La calidad del software no depende de un solo factor, sino de una "tecnología estratificada" que requiere un enfoque de calidad, procesos definidos, métodos específicos y el apoyo de herramientas CASE para automatizar tareas.

3. El desarrollo de software profesional debe cumplir con atributos esenciales como la mantenibilidad, confiabilidad, eficiencia y aceptabilidad, entendiendo que el costo de evolución y mantenimiento suele superar al de desarrollo inicial en sistemas de larga duración.

4. Existen diferencias fundamentales en los paradigmas de desarrollo; mientras el modelo en cascada entrega el producto al final, el modelo incremental permite entregas funcionales rápidas (núcleos) que evolucionan con los requisitos del usuario.

## RECOMENDACIONES

1. Seleccionar el paradigma de desarrollo (cascada, incremental o evolutivo) basándose en la estabilidad de los requisitos iniciales y la necesidad de entregas tempranas al cliente para reducir el tiempo de comercialización.

2. Priorizar las prácticas de desarrollo eficientes y una planificación rigurosa desde las etapas iniciales, ya que las deficiencias en estas áreas incrementan drásticamente los costos cuando el sistema ya está en operación.

3. Fortalecer la etapa de validación y pruebas, especialmente en sistemas críticos, destinando al menos el 40% del presupuesto para garantizar que el software sea seguro y resistente a fallos dañinos o intrusiones.

4. Utilizar marcos de trabajo estandarizados y normas internacionales, como la ISO/IEC 25010, para evaluar sistemáticamente características principales como la eficiencia de desempeño y la usabilidad del producto final.

### Integrantes Grupo 6

| |
|---|
| Vicente Mateo Sánchez Andrade |
| Juan Esteban Socasi Tenorio |
| Jonathan Andrés Rodríguez Nazareno |
| Joel Alejandro Tapia Suarez |

Sangolquí, a 11 de abril de 2026